



JavaScript Documentation

A practical reference for core JavaScript, ES6+, the DOM, and asynchronous programming

1. What is JavaScript?

JavaScript is a scripting language used to add interactivity and logic to web pages, and also runs outside the browser (Node.js) for backend/server code.

HTML → Structure
CSS → Styling
JS → Behavior / Logic

Where JS runs

Environment	Purpose
Browser	DOM manipulation, events, UI logic
Node.js	Backend servers, CLI tools, scripts
V8 / SpiderMonkey	JS engines that execute the code

2. Including JavaScript

Inline in HTML:

```
<script>
  console.log("Hello");
</script>
```

External file:

```
<script src="app.js"></script>
```

Defer execution until HTML is parsed:

```
<script src="app.js" defer></script>
```

3. Variables

Keyword	Reassignable	Scope
var	Yes	Function-scoped (legacy)
let	Yes	Block-scoped
const	No	Block-scoped

```
let age = 21;
const name = "Harsha";
var oldStyle = "avoid this";
```

⚠ Prefer `const` by default, and `let` only when reassignment is needed. Avoid `var`.

4. Data Types

Primitive types:

```
let str  = "text";      // string
let num  = 42;          // number
let bool = true;        // boolean
let undef;              // undefined
let empty = null;       // null
let sym  = Symbol("id"); // symbol
let big  = 10n;          // bigint
```

Reference types:

```
let arr = [1, 2, 3];      // array
let obj = { key: "value" }; // object
let fn  = function() {};  // function
```

Check type:

```
typeof "hello"; // "string"
typeof 42;       // "number"
typeof undefined; // "undefined"
Array.isArray([1,2,3]); // true
```

5. Operators

Arithmetic:

```
+ - * / % **
```

Comparison:

```
== // loose equality (avoid)
=== // strict equality (preferred)
!== // strict inequality
> < >= <=
```

Logical:

```
&& // AND
|| // OR
! // NOT
?? // Nullish coalescing
```

Assignment shortcuts:

```
let x = 5;
x += 2; // 7
x -= 1; // 6
x *= 2; // 12
```

Ternary:

```
let status = age >= 18 ? "adult" : "minor";
```

6. Strings

```
let name = "Harsha";

name.length;           // 6
name.toUpperCase();     // "HARSHA"
name.toLowerCase();     // "harsha"
name.includes("ars");   // true
name.slice(0, 3);       // "Har"
name.split("");         // ["H","a","r","s","h","a"]
name.trim();            // removes whitespace
name.replace("H", "J"); // "Jarsha"
```

Template literals:

```
let age = 21;
console.log(`My name is ${name} and I am ${age} years old.`);
```

7. Arrays

```
let fruits = ["apple", "banana", "mango"];

fruits.push("orange");    // add to end
fruits.pop();             // remove from end
fruits.shift();           // remove from start
fruits.unshift("kiwi");   // add to start
fruits.length;            // 3
fruits.indexOf("banana"); // 1
fruits.includes("mango"); // true
```

Iteration methods:

```
fruits.forEach(f => console.log(f));

let upper = fruits.map(f => f.toUpperCase());

let long = fruits.filter(f => f.length > 5);

let total = [1, 2, 3].reduce((sum, n) => sum + n, 0); // 6

let found = fruits.find(f => f.startsWith("m"));

let allLong = fruits.every(f => f.length > 3);
let anyLong = fruits.some(f => f.length > 5);
```

Sorting and slicing:

```
fruits.sort();
fruits.reverse();
fruits.slice(0, 2); // returns a copy, doesn't mutate
fruits.splice(1, 1); // removes/inserts, mutates original
```

8. Objects

```
const user = {
  name: "Harsha",
  age: 21,
  isStudent: true,
  greet() {
    console.log(`Hi, I'm ${this.name}`);
  }
};

user.name;           // dot notation
user["age"];         // bracket notation
user.greet();        // calling a method
```

Object methods:

```
Object.keys(user);    // ["name", "age", "isStudent", "greet"]
Object.values(user);  // values array
Object.entries(user); // [key, value] pairs
```

Spread and destructuring:

```
const { name, age } = user;           // destructuring
const copy = { ...user, age: 22 };    // spread + override

const arr = [1, 2, 3];
const [first, second] = arr;          // array destructuring
const combined = [...arr, 4, 5];      // spread
```

9. Functions

Declaration:

```
function add(a, b) {
  return a + b;
}
```

Expression:

```
const add = function(a, b) {
  return a + b;
};
```

Arrow function:

```
const add = (a, b) => a + b;

const square = x => x * x;           // single param, no parens needed
const greet = () => "Hello!";        // no params
```

Default parameters:

```
function greet(name = "Guest") {
  console.log(`Hello, ${name}`);
}
```

Rest parameters:

```
function sum(...numbers) {
  return numbers.reduce((a, b) => a + b, 0);
}

sum(1, 2, 3, 4); // 10
```

Arrow functions do not have their own `this` — they inherit it from the enclosing scope. This matters when using `this` inside object methods or class methods.

10. Control Flow

Conditionals:

```
if (age >= 18) {
  console.log("Adult");
} else if (age >= 13) {
  console.log("Teenager");
} else {
  console.log("Child");
}
```

Switch:

```
switch (day) {
  case "Mon":
    console.log("Start of week");
    break;
  case "Fri":
    console.log("Almost weekend");
    break;
  default:
    console.log("Regular day");
}
```

Loops:

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}

for (const fruit of fruits) {
  console.log(fruit);
}

for (const key in user) {
  console.log(key, user[key]);
}

let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}
```

11. The DOM (Document Object Model)

The DOM is the browser's in-memory tree representation of the HTML page.

```
document
├── html
│   ├── head
│   └── body
│       ├── div
│       ├── h1
│       └── p
```

Selecting elements:

```
document.getElementById("title");
document.querySelector(".card");      // first match
document.querySelectorAll(".card");  // all matches (NodeList)
document.getElementsByClassName("card");
document.getElementsByTagName("div");
```

Modifying elements:

```
const el = document.querySelector("#title");

el.textContent = "New Title";
el.innerHTML = "<span>Styled</span>";
el.style.color = "blue";
el.classList.add("active");
el.classList.remove("hidden");
el.classList.toggle("open");
el.setAttribute("data-id", "42");
```

Creating and inserting elements:

```
const div = document.createElement("div");
div.textContent = "New element";
document.body.appendChild(div);

el.remove(); // remove an element
```

12. Events

```
const btn = document.querySelector("#submit");

btn.addEventListener("click", () => {
  console.log("Button clicked");
});

btn.addEventListener("click", function(event) {
  event.preventDefault(); // stop default behavior
  event.stopPropagation(); // stop bubbling to parent
});
```

Common events:

```
click, dblclick, submit, keydown, keyup,
input, change, mouseover, mouseout,
DOMContentLoaded, load, resize, scroll
```

Event delegation:

```
document.querySelector("#list").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") {
    console.log("List item clicked:", e.target.textContent);
  }
});
```

13. Asynchronous JavaScript

JavaScript is single-threaded but uses an event loop to handle async operations without blocking.

Callbacks (legacy style):

```
function fetchData(callback) {
  setTimeout(() => {
    callback("data loaded");
  }, 1000);
}

fetchData((result) => console.log(result));
```

Promises:

```
const promise = new Promise((resolve, reject) => {
  const success = true;
  if (success) resolve("Done!");
  else reject("Error!");
});

promise
  .then(result => console.log(result))
  .catch(error => console.error(error))
  .finally(() => console.log("Finished"));
```

Async/Await (preferred style):

```
async function getData() {
  try {
    const response = await fetch("https://api.example.com/data");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("Fetch failed:", error);
  }
}
```

Running promises in parallel:

```
const [users, posts] = await Promise.all([
  fetch("/users").then(r => r.json()),
  fetch("/posts").then(r => r.json())
]);
```

14. Fetch API

```
// GET request
fetch("https://api.example.com/users")
  .then(res => res.json())
  .then(data => console.log(data));

// POST request
fetch("https://api.example.com/users", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Harsha" })
})
  .then(res => res.json())
  .then(data => console.log(data));
```

15. ES6+ Features

Modules:

```
// utils.js
export const add = (a, b) => a + b;
export default function greet() {}

// main.js
import greet, { add } from "./utils.js";
```

Classes:

```

class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a sound`);
  }
}

class Dog extends Animal {
  speak() {
    console.log(`${this.name} barks`);
  }
}

const d = new Dog("Rex");
d.speak(); // "Rex barks"

```

Optional chaining & nullish coalescing:

```

const city = user?.address?.city;      // undefined if any part is missing
const port = process.env.PORT ?? 3000; // fallback only if null/undefined

```

Map and Set:

```

const map = new Map();
map.set("name", "Harsha");
map.get("name");    // "Harsha"

const set = new Set([1, 2, 2, 3]);
set.size;           // 3 (duplicates removed)

```

16. Error Handling

```

try {
  JSON.parse("invalid json");
} catch (error) {
  console.error("Something went wrong:", error.message);
} finally {
  console.log("Always runs");
}

throw new Error("Custom error message");

```

17. JSON

```

const obj = { name: "Harsha", age: 21 };

const json = JSON.stringify(obj);    // object → JSON string
const parsed = JSON.parse(json);     // JSON string → object

```

18. `this` Keyword

```
const obj = {
  name: "Harsha",
  regular: function() {
    console.log(this.name); // "Harsha" – depends on how it's called
  },
  arrow: () => {
    console.log(this.name); // undefined – inherits outer scope
  }
};
```

Context

this refers to

Global scope	<code>window</code> (browser) / <code>undefined</code> (strict/module)
Object method	The object calling the method
Arrow function	Enclosing lexical scope
Event handler	The element that received the event
<code>new</code> constructor	The newly created instance

19. Node.js Basics

```
// Import a module
const fs = require("fs");           // CommonJS
import fs from "fs";                // ES Modules

// Reading a file
fs.readFile("data.txt", "utf8", (err, data) => {
  if (err) throw err;
  console.log(data);
});

// Environment variables
console.log(process.env.PORT);

// Simple server (with Express)
import express from "express";
const app = express();

app.get("/", (req, res) => {
  res.send("Hello World");
});

app.listen(5000, () => console.log("Server running on port 5000"));
```

20. Common Array/Object Patterns Cheat Sheet

Transform an array:

```
const doubled = numbers.map(n => n * 2);
```

Filter and count:

```
const adults = users.filter(u => u.age >= 18).length;
```

Group by key:

```
const grouped = items.reduce((acc, item) => {
  (acc[item.category] ??= []).push(item);
  return acc;
}, {});
```

Merge objects:

```
const merged = { ...defaults, ...userSettings };
```

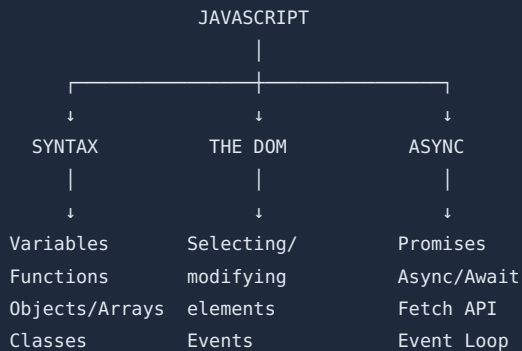
Remove duplicates:

```
const unique = [...new Set(array)];
```

Deep clone (simple cases):

```
const clone = structuredClone(original);
```

★ Core JavaScript Concepts



Recommended learning order

1. Variables, data types, operators
2. Functions & control flow
3. Arrays & objects (+ destructuring/spread)
4. DOM selection & manipulation
5. Events & event delegation
6. Asynchronous JS: callbacks → promises → async/await
7. Fetch API & working with JSON
8. ES6+ features: modules, classes, optional chaining
9. Error handling
10. Node.js fundamentals (modules, `process`, simple servers)

Understanding these gives the foundation to move from **JavaScript** → **Frameworks (React/Vue)** → **Full-stack development (Node.js/Express, FastAPI integration)**.